

Section 85 Editorial — Weighted And General Matching

A. Assignment Cost

How to find it

Maintain feasible Hungarian potentials and augment only through equality edges.

Correctness

Reduced costs stay nonnegative, giving a dual lower bound. The final perfect matching uses tight edges and reaches the bound.

Complexity

The reference follows the standard polynomial bound of the named augmentation, flow, or contraction algorithm; explicit problem limits keep oracle-based matroid intersection practical.

Here the rectangular Hungarian implementation runs in $O(n^2 m)$ for n rows and m columns, with $n \leq m$, and uses $O(n+m)$ auxiliary storage. The potentials are a verifiable optimality certificate: every reduced cost is nonnegative and every selected edge is tight.

C++

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
pair<ll, vector<int>>> hungarian(vector<vector<ll>>> a) {
    int n = a.size(), m = a[0].size();
    vector<ll> u(n + 1), v(m + 1);
    vector<int> p(m + 1), way(m + 1);
    for (int i = 1; i <= n; i++) {
        p[0] = i;
        int j0 = 0;
        vector<ll> mn(m + 1, 4e18);
        vector<char> used(m + 1);
        do {
            used[j0] = 1;
            int i0 = p[j0], j1 = 0;
            ll delta = 4e18;
            for (int j = 1; j <= m; j++)
                if (!used[j]) {
                    ll cur = a[i0 - 1][j - 1] - u[i0] - v[j];
                    if (cur < mn[j])
                        mn[j] = cur, way[j] = j0;
                    if (mn[j] < delta)
                        delta = mn[j], j1 = j;
                }
            for (int j = 0; j <= m; j++)
                if (used[j])
                    u[p[j]] += delta, v[j] -= delta;
            else
                mn[j] -= delta;
        } while (j1 != j0);
        p[j0] = p[j1];
    }
    vector<int> m1(m);
    for (int i = 1; i <= m; i++) m1[i - 1] = p[i];
    return {0, m1};
}
```

```

    j0 = j1;
} while (p[j0]);
do {
    int j1 = way[j0];
    p[j0] = p[j1];
    j0 = j1;
} while (j0);
}
vector<int> match(n);
for (int j = 1; j <= m; j++)
    if (p[j])
        match[p[j] - 1] = j - 1;
return {-v[0], match};
}

struct Blossom {
    int n;
    vector<vector<int>> g;
    vector<int> match, p, base, q;
    vector<char> used, flower;
    Blossom(vector<vector<int>> g)
        : n(g.size()), g(g), match(n, -1), p(n), base(n), q(n), used(n),
          flower(n) {}
    int lca(int a, int b) {
        vector<char> mark(n);
        while (1) {
            a = base[a];
            mark[a] = 1;
            if (match[a] < 0)
                break;
            a = p[match[a]];
        }
        while (1) {
            b = base[b];
            if (mark[b])
                return b;
            b = p[match[b]];
        }
    }
    void mark_path(int v, int b, int child) {
        while (base[v] != b) {
            flower[base[v]] = flower[base[match[v]]] = 1;
            p[v] = child;
            child = match[v];
            v = p[match[v]];
        }
    }
    int path(int root) {
        fill(used.begin(), used.end(), 0);
        fill(p.begin(), p.end(), -1);
        iota(base.begin(), base.end(), 0);
        int qh = 0, qt = 0;
        q[qt++] = root;
        used[root] = 1;
        while (qh < qt) {
            int v = q[qh++];
            for (int u : g[v])
                if (base[v] != base[u] && match[v] != u) {
                    if (u == root || (match[u] >= 0 && p[match[u]] >= 0)) {
                        int b = lca(v, u);
                        fill(flower.begin(), flower.end(), 0);
                        mark_path(v, b, u);
                        mark_path(u, b, v);
                        for (int x = 0; x < n; x++)

```

```

        if (flower[base[x]]) {
            base[x] = b;
            if (!used[x])
                used[x] = 1, q[qt++] = x;
        }
    } else if (p[u] < 0) {
        p[u] = v;
        if (match[u] < 0)
            return u;
        u = match[u];
        used[u] = 1;
        q[qt++] = u;
    }
}
}
return -1;
}

int solve() {
    for (int root = 0; root < n; root++)
        if (match[root] < 0) {
            int v = path(root);
            while (v >= 0) {
                int pv = p[v], nv = pv < 0 ? -1 : match[pv];
                match[v] = pv;
                if (pv >= 0)
                    match[pv] = v;
                v = nv;
            }
        }
    return count_if(match.begin(), match.end(), [](int x) { return x >= 0; }) /
        2;
}
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<vector<ll>> a(n, vector<ll>(n));
    for (auto &r : a)
        for (auto &x : r)
            cin >> x;
    cout << hungarian(a).first << '\n';
}

```

Python

```

import sys

def hungarian(cost):
    n = len(cost)
    m = len(cost[0])
    u = [0] * (n + 1)
    v = [0] * (m + 1)
    p = [0] * (m + 1)
    way = [0] * (m + 1)
    for i in range(1, n + 1):
        p[0] = i
        j0 = 0
        minimum = [10**30] * (m + 1)

```

```

used = [False] * (m + 1)
while True:
    used[j0] = True
    i0 = p[j0]
    delta = 10**30
    j1 = 0
    for j in range(1, m + 1):
        if not used[j]:
            cur = cost[i0 - 1][j - 1] - u[i0] - v[j]
            if cur < minimum[j]:
                minimum[j] = cur
                way[j] = j0
            if minimum[j] < delta:
                delta = minimum[j]
                j1 = j
    for j in range(m + 1):
        if used[j]:
            u[p[j]] += delta
            v[j] -= delta
        else:
            minimum[j] -= delta
    j0 = j1
    if p[j0] == 0:
        break
    while True:
        j1 = way[j0]
        p[j0] = p[j1]
        j0 = j1
        if j0 == 0:
            break
match = [-1] * n
for j in range(1, m + 1):
    if p[j]:
        match[p[j] - 1] = j - 1
return -v[0], match

```

```

def blossom(graph):
    n = len(graph)
    match = [-1] * n
    base = list(range(n))
    parent = [-1] * n

    def find_path(root):
        nonlocal base, parent
        used = [False] * n
        parent = [-1] * n
        base = list(range(n))
        q = [root]
        used[root] = True

    def get_lca(a, b):
        mark = [False] * n
        while True:
            a = base[a]
            mark[a] = True
            if match[a] == -1:
                break
            a = parent[match[a]]
        while True:
            b = base[b]
            if mark[b]:
                return b

```

```

        b = parent[match[b]]

    def mark_path(v, b, child, flower):
        while base[v] != b:
            flower[base[v]] = flower[base[match[v]]] = True
            parent[v] = child
            child = match[v]
            v = parent[match[v]]

    for v in q:
        for u in graph[v]:
            if base[v] == base[u] or match[v] == u:
                continue
            if u == root or (match[u] != -1 and parent[match[u]] != -1):
                b = get_lca(v, u)
                flower = [False] * n
                mark_path(v, b, u, flower)
                mark_path(u, b, v, flower)
                for x in range(n):
                    if flower[base[x]]:
                        base[x] = b
                    if not used[x] and flower[base[x]]:
                        used[x] = True
                        q.append(x)
            elif parent[u] == -1:
                parent[u] = v
                if match[u] == -1:
                    return u
                u = match[u]
                used[u] = True
                q.append(u)

    return -1

for root in range(n):
    if match[root] != -1:
        continue
    v = find_path(root)
    if v == -1:
        continue
    while v != -1:
        pv = parent[v]
        nv = match[pv] if pv != -1 else -1
        match[v] = pv
        if pv != -1:
            match[pv] = v
        v = nv
    return match

def main():
    d = list(map(int, sys.stdin.buffer.read().split()))
    n = d[0]
    a = [d[1 + i * n : 1 + (i + 1) * n] for i in range(n)]
    print(hungarian(a)[0])

if __name__ == "__main__":
    main()

```

B. Forbidden Profit Assignment

How to find it

Negate profits and encode forbidden pairs by a dominant infinity.

Correctness

Finite assignments preserve reversed objective order; an infinity edge occurs exactly when no complete allowed assignment exists.

Complexity

The reference follows the standard polynomial bound of the named augmentation, flow, or contraction algorithm; explicit problem limits keep oracle-based matroid intersection practical.

The sentinel must exceed the largest possible difference between two finite assignment totals. After Hungarian finishes, inspect the selected original pairs; a sentinel in the result means that no all-allowed complete assignment exists. The running time is $O(n^2 m)$.

C++

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
pair<ll, vector<int>> hungarian(vector<vector<ll>> a) {
    int n = a.size(), m = a[0].size();
    vector<ll> u(n + 1), v(m + 1);
    vector<int> p(m + 1), way(m + 1);
    for (int i = 1; i <= n; i++) {
        p[0] = i;
        int j0 = 0;
        vector<ll> mn(m + 1, 4e18);
        vector<char> used(m + 1);
        do {
            used[j0] = 1;
            int i0 = p[j0], j1 = 0;
            ll delta = 4e18;
            for (int j = 1; j <= m; j++)
                if (!used[j]) {
                    ll cur = a[i0 - 1][j - 1] - u[i0] - v[j];
                    if (cur < mn[j])
                        mn[j] = cur, way[j] = j0;
                    if (mn[j] < delta)
                        delta = mn[j], j1 = j;
                }
            for (int j = 0; j <= m; j++)
                if (used[j])
                    u[p[j]] += delta, v[j] -= delta;
            else
                mn[j] -= delta;
            j0 = j1;
        } while (p[j0]);
        do {
            int j1 = way[j0];
            p[j0] = p[j1];
            j0 = j1;
        } while (j0);
    }
    vector<int> match(n);
    for (int j = 1; j <= m; j++)
```

```

    if (p[j])
        match[p[j] - 1] = j - 1;
    return {-v[0], match};
}

struct Blossom {
    int n;
    vector<vector<int>> g;
    vector<int> match, p, base, q;
    vector<char> used, flower;
    Blossom(vector<vector<int>> g)
        : n(g.size()), g(g), match(n, -1), p(n), base(n), q(n), used(n),
          flower(n) {}
    int lca(int a, int b) {
        vector<char> mark(n);
        while (1) {
            a = base[a];
            mark[a] = 1;
            if (match[a] < 0)
                break;
            a = p[match[a]];
        }
        while (1) {
            b = base[b];
            if (mark[b])
                return b;
            b = p[match[b]];
        }
    }
    void mark_path(int v, int b, int child) {
        while (base[v] != b) {
            flower[base[v]] = flower[base[match[v]]] = 1;
            p[v] = child;
            child = match[v];
            v = p[match[v]];
        }
    }
    int path(int root) {
        fill(used.begin(), used.end(), 0);
        fill(p.begin(), p.end(), -1);
        iota(base.begin(), base.end(), 0);
        int qh = 0, qt = 0;
        q[qt++] = root;
        used[root] = 1;
        while (qh < qt) {
            int v = q[qh++];
            for (int u : g[v])
                if (base[v] != base[u] && match[v] != u) {
                    if (u == root || (match[u] >= 0 && p[match[u]] >= 0)) {
                        int b = lca(v, u);
                        fill(flower.begin(), flower.end(), 0);
                        mark_path(v, b, u);
                        mark_path(u, b, v);
                        for (int x = 0; x < n; x++)
                            if (flower[base[x]]) {
                                base[x] = b;
                                if (!used[x])
                                    used[x] = 1, q[qt++] = x;
                            }
                    }
                    else if (p[u] < 0) {
                        p[u] = v;
                        if (match[u] < 0)
                            return u;
                        u = match[u];
                    }
                }
        }
    }
};

```

```

        used[u] = 1;
        q[qt++] = u;
    }
}
return -1;
}
int solve() {
    for (int root = 0; root < n; root++)
        if (match[root] < 0) {
            int v = path(root);
            while (v >= 0) {
                int pv = p[v], nv = pv < 0 ? -1 : match[pv];
                match[v] = pv;
                if (pv >= 0)
                    match[pv] = v;
                v = nv;
            }
        }
    return count_if(match.begin(), match.end(), [](int x) { return x >= 0; }) /
        2;
}
};
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    const ll X = 1e15;
    vector<vector<ll>> a(n, vector<ll>(m));
    for (auto &r : a)
        for (auto &x : r) {
            cin >> x;
            x = x < 0 ? X : -x;
        }
    auto [cost, match] = hungarian(a);
    for (int i = 0; i < n; i++)
        if (a[i][match[i]] >= X) {
            cout << "IMPOSSIBLE\n";
            return 0;
        }
    cout << -cost << '\n';
}

```

Python

```

import sys

def hungarian(cost):
    n = len(cost)
    m = len(cost[0])
    u = [0] * (n + 1)
    v = [0] * (m + 1)
    p = [0] * (m + 1)
    way = [0] * (m + 1)
    for i in range(1, n + 1):
        p[0] = i
        j0 = 0
        minimum = [10**30] * (m + 1)
        used = [False] * (m + 1)

```

```

while True:
    used[j0] = True
    i0 = p[j0]
    delta = 10**30
    j1 = 0
    for j in range(1, m + 1):
        if not used[j]:
            cur = cost[i0 - 1][j - 1] - u[i0] - v[j]
            if cur < minimum[j]:
                minimum[j] = cur
                way[j] = j0
            if minimum[j] < delta:
                delta = minimum[j]
                j1 = j
    for j in range(m + 1):
        if used[j]:
            u[p[j]] += delta
            v[j] -= delta
        else:
            minimum[j] -= delta
    j0 = j1
    if p[j0] == 0:
        break
    while True:
        j1 = way[j0]
        p[j0] = p[j1]
        j0 = j1
        if j0 == 0:
            break
match = [-1] * n
for j in range(1, m + 1):
    if p[j]:
        match[p[j] - 1] = j - 1
return -v[0], match

```

```

def blossom(graph):
    n = len(graph)
    match = [-1] * n
    base = list(range(n))
    parent = [-1] * n

    def find_path(root):
        nonlocal base, parent
        used = [False] * n
        parent = [-1] * n
        base = list(range(n))
        q = [root]
        used[root] = True

    def get_lca(a, b):
        mark = [False] * n
        while True:
            a = base[a]
            mark[a] = True
            if match[a] == -1:
                break
            a = parent[match[a]]
        while True:
            b = base[b]
            if mark[b]:
                return b
            b = parent[match[b]]

```

```

def mark_path(v, b, child, flower):
    while base[v] != b:
        flower[base[v]] = flower[base[match[v]]] = True
        parent[v] = child
        child = match[v]
        v = parent[match[v]]

for v in q:
    for u in graph[v]:
        if base[v] == base[u] or match[v] == u:
            continue
        if u == root or (match[u] != -1 and parent[match[u]] != -1):
            b = get_lca(v, u)
            flower = [False] * n
            mark_path(v, b, u, flower)
            mark_path(u, b, v, flower)
            for x in range(n):
                if flower[base[x]]:
                    base[x] = b
                if not used[x] and flower[base[x]]:
                    used[x] = True
                    q.append(x)
            elif parent[u] == -1:
                parent[u] = v
                if match[u] == -1:
                    return u
                u = match[u]
                used[u] = True
                q.append(u)

return -1

for root in range(n):
    if match[root] != -1:
        continue
    v = find_path(root)
    if v == -1:
        continue
    while v != -1:
        pv = parent[v]
        nv = match[pv] if pv != -1 else -1
        match[v] = pv
        if pv != -1:
            match[pv] = v
        v = nv
return match

def main():
    d = list(map(int, sys.stdin.buffer.read().split()))
    n, m = d[:2]
    raw = d[2:]
    inf = 10**15
    a = [
        [inf if raw[i * m + j] < 0 else -raw[i * m + j] for j in range(m)]
        for i in range(n)
    ]
    cost, match = hungarian(a)
    print(
        "IMPOSSIBLE"
        if any(a[i][match[i]] >= inf for i in range(n))
        else -cost
    )

```

```
if __name__ == "__main__":
    main()
```

C. General Pairing

How to find it

Contract odd alternating cycles between even BFS vertices.

Correctness

Blossom contraction preserves augmenting-path existence, and expansion of predecessors yields a valid original path whose flip grows the matching.

Complexity

The reference follows the standard polynomial bound of the named augmentation, flow, or contraction algorithm; explicit problem limits keep oracle-based matroid intersection practical.

With the adjacency-matrix-style blossom implementation, at most n augmentations each take $O(n^2)$, for $O(n^3)$ time and $O(n^2)$ graph storage. A triangle, a blossom with a tail, and nested odd cycles are the minimum useful structural tests.

C++

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
pair<ll, vector<int>> hungarian(vector<vector<ll>> a) {
    int n = a.size(), m = a[0].size();
    vector<ll> u(n + 1), v(m + 1);
    vector<int> p(m + 1), way(m + 1);
    for (int i = 1; i <= n; i++) {
        p[0] = i;
        int j0 = 0;
        vector<ll> mn(m + 1, 4e18);
        vector<char> used(m + 1);
        do {
            used[j0] = 1;
            int i0 = p[j0], j1 = 0;
            ll delta = 4e18;
            for (int j = 1; j <= m; j++)
                if (!used[j]) {
                    ll cur = a[i0 - 1][j - 1] - u[i0] - v[j];
                    if (cur < mn[j])
                        mn[j] = cur, way[j] = j0;
                    if (mn[j] < delta)
                        delta = mn[j], j1 = j;
                }
            for (int j = 0; j <= m; j++)
                if (used[j])
                    u[p[j]] += delta, v[j] -= delta;
                else
                    mn[j] -= delta;
            j0 = j1;
        } while (p[j0] == 0);
        do {
            int j1 = way[j0];
```

```

    p[j0] = p[j1];
    j0 = j1;
} while (j0);
}
vector<int> match(n);
for (int j = 1; j <= m; j++)
    if (p[j])
        match[p[j] - 1] = j - 1;
return {-v[0], match};
}

struct Blossom {
    int n;
    vector<vector<int>> g;
    vector<int> match, p, base, q;
    vector<char> used, flower;
    Blossom(vector<vector<int>> g)
        : n(g.size()), g(g), match(n, -1), p(n), base(n), q(n), used(n),
          flower(n) {}
    int lca(int a, int b) {
        vector<char> mark(n);
        while (1) {
            a = base[a];
            mark[a] = 1;
            if (match[a] < 0)
                break;
            a = p[match[a]];
        }
        while (1) {
            b = base[b];
            if (mark[b])
                return b;
            b = p[match[b]];
        }
    }
    void mark_path(int v, int b, int child) {
        while (base[v] != b) {
            flower[base[v]] = flower[base[match[v]]] = 1;
            p[v] = child;
            child = match[v];
            v = p[match[v]];
        }
    }
    int path(int root) {
        fill(used.begin(), used.end(), 0);
        fill(p.begin(), p.end(), -1);
        iota(base.begin(), base.end(), 0);
        int qh = 0, qt = 0;
        q[qt++] = root;
        used[root] = 1;
        while (qh < qt) {
            int v = q[qh++];
            for (int u : g[v])
                if (base[v] != base[u] && match[v] != u) {
                    if (u == root || (match[u] >= 0 && p[match[u]] >= 0)) {
                        int b = lca(v, u);
                        fill(flower.begin(), flower.end(), 0);
                        mark_path(v, b, u);
                        mark_path(u, b, v);
                        for (int x = 0; x < n; x++)
                            if (flower[base[x]]) {
                                base[x] = b;
                                if (!used[x])
                                    used[x] = 1, q[qt++] = x;
                            }
                    }
                }
        }
    }
};

```

```

        }
    } else if (p[u] < 0) {
        p[u] = v;
        if (match[u] < 0)
            return u;
        u = match[u];
        used[u] = 1;
        q[qt++] = u;
    }
}

return -1;
}

int solve() {
    for (int root = 0; root < n; root++)
        if (match[root] < 0) {
            int v = path(root);
            while (v >= 0) {
                int pv = p[v], nv = pv < 0 ? -1 : match[pv];
                match[v] = pv;
                if (pv >= 0)
                    match[pv] = v;
                v = nv;
            }
        }

    return count_if(match.begin(), match.end(), [](int x) { return x >= 0; }) /
           2;
}

};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<vector<int>> g(n);
    while (m--) {
        int u, v;
        cin >> u >> v;
        --u;
        --v;
        g[u].push_back(v);
        g[v].push_back(u);
    }
    cout << Blossom(g).solve() << '\n';
}

```

Python

```

import sys

def hungarian(cost):
    n = len(cost)
    m = len(cost[0])
    u = [0] * (n + 1)
    v = [0] * (m + 1)
    p = [0] * (m + 1)
    way = [0] * (m + 1)
    for i in range(1, n + 1):
        p[0] = i
        j0 = 0

```

```

minimum = [10**30] * (m + 1)
used = [False] * (m + 1)
while True:
    used[j0] = True
    i0 = p[j0]
    delta = 10**30
    j1 = 0
    for j in range(1, m + 1):
        if not used[j]:
            cur = cost[i0 - 1][j - 1] - u[i0] - v[j]
            if cur < minimum[j]:
                minimum[j] = cur
                way[j] = j0
            if minimum[j] < delta:
                delta = minimum[j]
                j1 = j
    for j in range(m + 1):
        if used[j]:
            u[p[j]] += delta
            v[j] -= delta
        else:
            minimum[j] -= delta
    j0 = j1
    if p[j0] == 0:
        break
    while True:
        j1 = way[j0]
        p[j0] = p[j1]
        j0 = j1
        if j0 == 0:
            break
match = [-1] * n
for j in range(1, m + 1):
    if p[j]:
        match[p[j] - 1] = j - 1
return -v[0], match

```

```

def blossom(graph):
    n = len(graph)
    match = [-1] * n
    base = list(range(n))
    parent = [-1] * n

    def find_path(root):
        nonlocal base, parent
        used = [False] * n
        parent = [-1] * n
        base = list(range(n))
        q = [root]
        used[root] = True

    def get_lca(a, b):
        mark = [False] * n
        while True:
            a = base[a]
            mark[a] = True
            if match[a] == -1:
                break
            a = parent[match[a]]
        while True:
            b = base[b]
            if mark[b]:

```

```

        return b
    b = parent[match[b]]

def mark_path(v, b, child, flower):
    while base[v] != b:
        flower[base[v]] = flower[base[match[v]]] = True
        parent[v] = child
        child = match[v]
        v = parent[match[v]]

for v in q:
    for u in graph[v]:
        if base[v] == base[u] or match[v] == u:
            continue
        if u == root or (match[u] != -1 and parent[match[u]] != -1):
            b = get_lca(v, u)
            flower = [False] * n
            mark_path(v, b, u, flower)
            mark_path(u, b, v, flower)
            for x in range(n):
                if flower[base[x]]:
                    base[x] = b
                if not used[x] and flower[base[x]]:
                    used[x] = True
                    q.append(x)
            elif parent[u] == -1:
                parent[u] = v
                if match[u] == -1:
                    return u
                u = match[u]
                used[u] = True
                q.append(u)

    return -1

for root in range(n):
    if match[root] != -1:
        continue
    v = find_path(root)
    if v == -1:
        continue
    while v != -1:
        pv = parent[v]
        nv = match[pv] if pv != -1 else -1
        match[v] = pv
        if pv != -1:
            match[pv] = v
        v = nv
    return match

def main():
    d = list(map(int, sys.stdin.buffer.read().split()))
    n, m = d[:2]
    g = [[] for _ in range(n)]
    for i in range(2, 2 + 2 * m, 2):
        u, v = d[i] - 1, d[i + 1] - 1
        g[u].append(v)
        g[v].append(u)
    matched = sum(x >= 0 for x in blossom(g)) // 2
    print(matched)

if __name__ == "__main__":

```

```
main()
```

D. Compatibility Pairing

How to find it

Build exactly the allowed compatibility graph and run general matching.

Correctness

Valid disjoint pair families are precisely graph matchings, so their optima coincide.

Complexity

The reference follows the standard polynomial bound of the named augmentation, flow, or contraction algorithm; explicit problem limits keep oracle-based matroid intersection practical.

Building the graph is linear in the declared compatible pairs; the blossom phase is $O(n^3)$. Check the parity of n , but do not confuse even size with guaranteed feasibility: the matching must still cover every vertex.

C++

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
pair<ll, vector<int>> hungarian(vector<vector<ll>> a) {
    int n = a.size(), m = a[0].size();
    vector<ll> u(n + 1), v(m + 1);
    vector<int> p(m + 1), way(m + 1);
    for (int i = 1; i <= n; i++) {
        p[0] = i;
        int j0 = 0;
        vector<ll> mn(m + 1, 4e18);
        vector<char> used(m + 1);
        do {
            used[j0] = 1;
            int i0 = p[j0], j1 = 0;
            ll delta = 4e18;
            for (int j = 1; j <= m; j++)
                if (!used[j]) {
                    ll cur = a[i0 - 1][j - 1] - u[i0] - v[j];
                    if (cur < mn[j])
                        mn[j] = cur, way[j] = j0;
                    if (mn[j] < delta)
                        delta = mn[j], j1 = j;
                }
            for (int j = 0; j <= m; j++)
                if (used[j])
                    u[p[j]] += delta, v[j] -= delta;
            else
                mn[j] -= delta;
            j0 = j1;
        } while (p[j0]);
        do {
            int j1 = way[j0];
            p[j0] = p[j1];
            j0 = j1;
        } while (j0);
    }
}
```

```

vector<int> match(n);
for (int j = 1; j <= m; j++)
    if (p[j])
        match[p[j] - 1] = j - 1;
return {-v[0], match};
}

struct Blossom {
    int n;
    vector<vector<int>> g;
    vector<int> match, p, base, q;
    vector<char> used, flower;
    Blossom(vector<vector<int>> g)
        : n(g.size()), g(g), match(n, -1), p(n), base(n), q(n), used(n),
          flower(n) {}
    int lca(int a, int b) {
        vector<char> mark(n);
        while (1) {
            a = base[a];
            mark[a] = 1;
            if (match[a] < 0)
                break;
            a = p[match[a]];
        }
        while (1) {
            b = base[b];
            if (mark[b])
                return b;
            b = p[match[b]];
        }
    }
    void mark_path(int v, int b, int child) {
        while (base[v] != b) {
            flower[base[v]] = flower[base[match[v]]] = 1;
            p[v] = child;
            child = match[v];
            v = p[match[v]];
        }
    }
    int path(int root) {
        fill(used.begin(), used.end(), 0);
        fill(p.begin(), p.end(), -1);
        iota(base.begin(), base.end(), 0);
        int qh = 0, qt = 0;
        q[qt++] = root;
        used[root] = 1;
        while (qh < qt) {
            int v = q[qh++];
            for (int u : g[v])
                if (base[v] != base[u] && match[v] != u) {
                    if (u == root || (match[u] >= 0 && p[match[u]] >= 0)) {
                        int b = lca(v, u);
                        fill(flower.begin(), flower.end(), 0);
                        mark_path(v, b, u);
                        mark_path(u, b, v);
                        for (int x = 0; x < n; x++)
                            if (flower[base[x]]) {
                                base[x] = b;
                                if (!used[x])
                                    used[x] = 1, q[qt++] = x;
                            }
                    }
                    else if (p[u] < 0) {
                        p[u] = v;
                        if (match[u] < 0)

```

```

        return u;
        u = match[u];
        used[u] = 1;
        q[qt++] = u;
    }
}
}
return -1;
}
int solve() {
    for (int root = 0; root < n; root++)
        if (match[root] < 0) {
            int v = path(root);
            while (v >= 0) {
                int pv = p[v], nv = pv < 0 ? -1 : match[pv];
                match[v] = pv;
                if (pv >= 0)
                    match[pv] = v;
                v = nv;
            }
        }
    return count_if(match.begin(), match.end(), [](int x) { return x >= 0; }) /
           2;
}
};
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, k;
    cin >> n >> k;
    vector<int> a(n);
    for (int &x : a)
        cin >> x;
    set<int> ok;
    while (k--) {
        int x;
        cin >> x;
        ok.insert(x);
    }
    vector<vector<int>> g(n);
    for (int i = 0; i < n; i++)
        for (int j = 0; j < i; j++)
            if (ok.count(abs(a[i] - a[j])))
                g[i].push_back(j), g[j].push_back(i);
    cout << Blossom(g).solve() << '\n';
}

```

Python

```

import sys

def hungarian(cost):
    n = len(cost)
    m = len(cost[0])
    u = [0] * (n + 1)
    v = [0] * (m + 1)
    p = [0] * (m + 1)
    way = [0] * (m + 1)
    for i in range(1, n + 1):
        p[0] = i
        j0 = 0

```

```

minimum = [10**30] * (m + 1)
used = [False] * (m + 1)
while True:
    used[j0] = True
    i0 = p[j0]
    delta = 10**30
    j1 = 0
    for j in range(1, m + 1):
        if not used[j]:
            cur = cost[i0 - 1][j - 1] - u[i0] - v[j]
            if cur < minimum[j]:
                minimum[j] = cur
                way[j] = j0
            if minimum[j] < delta:
                delta = minimum[j]
                j1 = j
    for j in range(m + 1):
        if used[j]:
            u[p[j]] += delta
            v[j] -= delta
        else:
            minimum[j] -= delta
    j0 = j1
    if p[j0] == 0:
        break
    while True:
        j1 = way[j0]
        p[j0] = p[j1]
        j0 = j1
        if j0 == 0:
            break
match = [-1] * n
for j in range(1, m + 1):
    if p[j]:
        match[p[j] - 1] = j - 1
return -v[0], match

```

```

def blossom(graph):
    n = len(graph)
    match = [-1] * n
    base = list(range(n))
    parent = [-1] * n

    def find_path(root):
        nonlocal base, parent
        used = [False] * n
        parent = [-1] * n
        base = list(range(n))
        q = [root]
        used[root] = True

    def get_lca(a, b):
        mark = [False] * n
        while True:
            a = base[a]
            mark[a] = True
            if match[a] == -1:
                break
            a = parent[match[a]]
        while True:
            b = base[b]
            if mark[b]:

```

```

        return b
        b = parent[match[b]]

def mark_path(v, b, child, flower):
    while base[v] != b:
        flower[base[v]] = flower[base[match[v]]] = True
        parent[v] = child
        child = match[v]
        v = parent[match[v]]

for v in q:
    for u in graph[v]:
        if base[v] == base[u] or match[v] == u:
            continue
        if u == root or (match[u] != -1 and parent[match[u]] != -1):
            b = get_lca(v, u)
            flower = [False] * n
            mark_path(v, b, u, flower)
            mark_path(u, b, v, flower)
            for x in range(n):
                if flower[base[x]]:
                    base[x] = b
                if not used[x] and flower[base[x]]:
                    used[x] = True
                    q.append(x)
            elif parent[u] == -1:
                parent[u] = v
                if match[u] == -1:
                    return u
                u = match[u]
                used[u] = True
                q.append(u)

return -1

for root in range(n):
    if match[root] != -1:
        continue
    v = find_path(root)
    if v == -1:
        continue
    while v != -1:
        pv = parent[v]
        nv = match[pv] if pv != -1 else -1
        match[v] = pv
        if pv != -1:
            match[pv] = v
        v = nv
return match

def main():
    d = list(map(int, sys.stdin.buffer.read().split()))
    n, k = d[:2]
    a = d[2 : 2 + n]
    allowed = set(d[2 + n :])
    g = [[] for _ in range(n)]
    for i in range(n):
        for j in range(i):
            if abs(a[i] - a[j]) in allowed:
                g[i].append(j)
                g[j].append(i)
    print(sum(x >= 0 for x in blossom(g)) // 2)

```

```
if __name__ == "__main__":  
    main()
```

Upsolving checklist

For assignment, compare with all permutations for tiny rectangular matrices. For general matching, enumerate every choice of disjoint edges. Include ties, negative profits, isolated vertices, parallel descriptions of the same pair, and cases in which the first locally attractive pair prevents a perfect matching. These oracles test the reduction and objective, not merely whether the produced pairs are disjoint.